

Relatório LLMs

Sophy dos Santos

9 de junho de 2026

0.1 Escolha da ferramenta

- nome da ferramenta utilizada: **Gemini**
- data de uso: **09/06**
- versão: **Gratuita**
- acesso a internet? **Sim**

0.2 Primeira tarefa: explicação conceitual

0.2.1 Peça para a LLM explicar um conceito importante da Computação. Utilize inicialmente um prompt simples.

Prompt: “O QUE É SQL AI DIABA?”

Resultado: “O termo **SQL AI** (ou *Inteligência Artificial aplicada a SQL*) refere-se à convergência entre sistemas de bancos de dados tradicionais e a inteligência artificial generativa.

Em termos simples: é a capacidade de um sistema de IA entender o que você quer saber em linguagem natural (português ou inglês, por exemplo) e traduzir isso automaticamente para uma consulta SQL complexa, executando-a no seu banco de dados.”

0.2.2 Crie um prompt mais elaborado contendo contexto, público-alvo e formato desejado.

Prompt: “Sou uma aluna de graduação em ciência da computação, estou no meu primeiro semestre e preciso de uma explicação resumida sobre o básico de SQL. faça como um texto para relatório”

Resultado:

“Relatório: Fundamentos de SQL (Structured Query Language)”

1. Introdução

O SQL é uma linguagem de domínio específico, padronizada pela ISO, projetada para o gerenciamento e a manipulação de dados armazenados em Sistemas de Gerenciamento de Bancos de Dados Relacionais (SGBDR). Diferente de linguagens de programação procedurais (como C ou Python), o SQL é uma **linguagem declarativa**: o usuário descreve o que deseja obter, deixando para o motor do banco de dados definir o como (plano de execução).

2. A Estrutura Relacional

A base do SQL reside no Modelo Relacional, onde os dados são organizados em estruturas chamadas **tabelas** (relações). Cada tabela é composta por:

- **Linhas (Tuplas/Registros):** Representam uma instância única de um dado.
- **Colunas (Atributos/Campos):** Definem os tipos de dados e características daquela entidade.

3. Principais Categorias de Comandos

As operações em SQL são tradicionalmente divididas em subconjuntos baseados em sua finalidade:

- **DML (Data Manipulation Language):** Focada na interação com os dados contidos nas tabelas.
 - **SELECT:** Recupera dados (o comando mais utilizado).
 - **INSERT:** Adiciona novos registros.
 - **UPDATE:** Modifica registros existentes.
 - **DELETE:** Remove registros.
- **DDL (Data Definition Language):** Utilizada para definir a estrutura (o "esqueleto") do banco.
 - **CREATE:** Cria novos bancos, tabelas ou índices.
 - **ALTER:** Modifica a estrutura de um objeto existente.
 - **DROP:** Exclui estruturas permanentemente.
- **DCL (Data Control Language):** Lida com permissões e segurança (ex: **GRANT** para conceder acesso, **REVOKE** para retirar).

4. A Anatomia de uma Consulta (O Comando SELECT)

A estrutura básica de uma consulta segue uma lógica de filtragem:

- **SELECT:** Define quais colunas serão retornadas.

- **FROM:** Indica a origem dos dados (a tabela).
- **WHERE:** Filtra os registros com base em condições lógicas.
- **JOIN:** Permite combinar dados de duas ou mais tabelas através de chaves relacionadas (Foreign Keys), um pilar da normalização de dados.

5. Considerações Técnicas

Como aluna de Ciência da Computação, é importante notar que o SQL moderno vai além da simples extração. Ele envolve conceitos como **Transações (ACID)** — garantindo que operações sejam atômicas, consistentes, isoladas e duráveis — e **Índices**, estruturas de dados (geralmente árvores B) que otimizam a complexidade de tempo de busca de $O(n)$ para $O(\log n)$.

Sugestão de estruturação para o seu relatório:

1. **Contextualização:** O papel do SQL como interface entre a aplicação e o dado.
2. **Desenvolvimento:** A distinção entre DDL e DML.
3. **Análise Crítica:** A importância das chaves primárias e estrangeiras na integridade referencial.
4. **Conclusão:** A importância da otimização de consultas (Performance) em sistemas de larga escala.

Este resumo cobre os tópicos essenciais que compõem o currículo básico de uma disciplina de Banco de Dados. Caso precise aprofundar em algum desses comandos específicos ou em lógica de normalização, posso detalhar conforme o seu progresso no semestre.”

0.2.3 Comparando as respostas

Obviamente, com mais informações e contexto, a resposta foi muito melhor, mais rica e direto ao ponto. Enquanto com linguagem menos concisa a LLM até mesmo não entendeu sobre o tópico que eu queria.

0.3 Segunda tarefa: geração de código

Prompt: faz um coiso de código ai pra um jogo de adivinhar um número

Resultado:

Código:

```
import random

def jogo_adivinacao():
    print("--- Bem-vindo ao Jogo de Adivinhação! ---")
    print("Estou pensando em um número entre 1 e 100.")

    # O computador escolhe um número aleatório
    numero_secreto = random.randint(1, 100)
    tentativas = 0
    acertou = False

    # Loop principal do jogo
    while not acertou:
        try:
            palpite = int(input("Qual é o seu palpite? "))
            tentativas += 1

            if palpite < numero_secreto:
                print("Muito baixo! Tente um número maior.")
            elif palpite > numero_secreto:
                print("Muito alto! Tente um número menor.")
            else:
                print(f"Parabéns! Você acertou em {tentativas} tentativas.")
                acertou = True
        except ValueError:
            print("Por favor, digite apenas números inteiros.")

    # Executar o jogo
    if __name__ == "__main__":
        jogo_adivinacao()
}
```

Explicação do código: A explicação a seguir foi feita sem nenhum prompt.

“ Explicação rápida para seus estudos:

1. **import random:** Importamos uma biblioteca padrão do Python para gerar números aleatórios.
2. **random.randint(1, 100):** Gera um número inteiro entre 1 e 100.
3. **while not acertou:** Este é o laço de repetição. O jogo continua perguntando ao usuário até que ele acerte.
4. **try / except:** Isso é um tratamento de exceção. Se o usuário digitar uma letra em vez de um número, o programa não trava; ele exibe uma mensagem de aviso e pede um novo valor.
5. **Condicionais (if/elif/else):** É a lógica que compara o palpite do usuário com o número gerado pelo sistema.

Você pode rodar esse código em qualquer ambiente Python (como o VS Code, PyCharm ou até online no Google Colab). Quer que eu adicione algum recurso extra, como um limite de tentativas ou um nível de dificuldade? “

0.3.1 Testes

Em primeira mão, tentei jogar o jogo normalmente, nada aconteceu de errado, então tentei forçar erros. Porém, o código tem salvaguardas para palavras, dando um alerta para qualquer valor diferente de um inteiro. O único “problema” que pude achar foi que mesmo se eu botasse um valor acima de 100 ou menor que zero ele mandava as mesmas mensagens do que se eu tivesse adivinhado dentro do intervalo... mas é algo bem nichado.

0.3.2 Análise Crítica

No fim, acredito que o código foi bem consistente, enquanto claro, a linguagem foi escolhida pela própria máquina e teve aquela questão do intervalo, mas em geral foi bem conciso em relação à expectativa que eu tinha.

0.4 Melhorando o prompt

Prompt: “Gostaria de um código, feito em C, para um jogo de adivinhação para um número aleatório. Crie barreiras contra erros para caso o jogador escolha um número sem sentido (fora do intervalo aleatório) e comente o código para uma pessoa de nível avançado em C compreender”

Código:

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <stdbool.h>
#include <limits.h>

/* * Função utilitária para limpar o buffer de entrada.
 * Essencial em C para evitar que caracteres residuais no stdin
 * corrompam a próxima leitura.
 */
void flush_stdin() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

int main(void) {
    // Seed para o gerador de números pseudoaleatórios
    srand((unsigned int)time(NULL));

    const int MIN = 1;
    const int MAX = 100;
    int numero_secreto = (rand() % (MAX - MIN + 1)) + MIN;
    int tentativa = 0;
    int palpite = 0;
    bool acertou = false;

    printf("--- Jogo de Adivinhacao (Nivel Avancado) ---\n");
    printf("Intervalo: [%d, %d]\n", MIN, MAX);

    while (!acertou) {
        printf("\nDigite seu palpite: ");

        /* * scanf retorna o numero de itens lidos com sucesso.
         * Se o usuario digitar letras, retorna 0.
         */
        if (scanf("%d", &palpite) != 1) {
            fprintf(stderr, "Erro: Entrada invalida. Por favor, insira um inteiro.\n");
            flush_stdin(); // Limpa o buffer de caracteres inválidos
            continue;
        }

        // Validacao de dominio (barreiras de logica de negocio)
        if (palpite < MIN || palpite > MAX) {
            printf("Aviso: O numero deve estar entre %d e %d.\n", MIN, MAX);
            continue;
        }

        tentativa++;

        // Avaliacao do palpite com logica otimizada
        if (palpite < numero_secreto) {
            printf("Muito baixo!\n");
        } else if (palpite > numero_secreto) {
            printf("Muito alto!\n");
        } else {
            acertou = true;
            printf("Parabens! Acertou em %d tentativas.\n", tentativa);
        }
    }

    return EXIT_SUCCESS;
}

```

0.4.1 Comparação

O código anterior em python não estava ruim, mas ele de fato melhorou com essa nova geração, o fail-save pro intervalo, o fato de que o código consta para os problemas de buffer que o C tem, também gosto como ele só incrementa as tentativas se for válida.

0.5 Verificação crítica

0.5.1 Escolha uma resposta da LLM e responda:

Vamos utilizar a primeira pergunta, que ela interpretou sobre “SQL AI”

1. A resposta estava correta?

Não. Ela interpretou a gíria “AI” como se fosse “Artificial Intelligence”. Mesmo seguindo essa lógica, SQL AI não é um “termo” amplamente usado... Existe um site com esse nome que se aplica ao contexto, mas a máquina fala como se fosse algo amplamente conhecido e utilizado, o que não é.

2. Havia informações incompletas ou incorretas?

Ela deu um resumo tecnicamente coeso... Mas não era o que foi perguntado, além de que foi extremamente raso. Porém tem muitas pesquisas sobre a utilização de SQL com AI, tanto formais quanto informais, que poderiam se enquadrar com o que o Gemini resultou, mas nada

3. Como a resposta foi verificada?

Pesquisei se existiam fontes que usavam o termo “SQL AI” como algo substancial, mas além do site que encontrei, nada parecia ter muita relação além de alguns sites e fóruns discutindo sobre a utilização de AI para códigos SQL.

4. A LLM demonstrou excesso de confiança?

Com certeza, inventou um termo e usou como se fosse algo amplamente conhecido, enquanto provavelmente não se passava de uma alucinação.

5. Como o prompt poderia ser melhorado?

Não utilizar gírias e deixar mais claro o que você busca, basicamente o que fizemos no segundo exercício.

0.6 Boas práticas de uso

1. Por que não devemos copiar respostas sem revisão?

LLMs tem o costume de alucinar informações, tentar forçar uma resposta mesmo que elas não saibam ou a pergunta nem sequer faça sentido, por isso é necessário sempre revisar e confirmar o que é dito.

2. Por que é importante informar os prompts utilizados?

Porque com diferentes prompts, a interpretação, filtro e contexto que a máquina vai receber é alterado. Prompts mais concisos e detalhados vão gerar melhores resultados do que algo que deixa brechas para a máquina alucinar.

3. Em quais situações as LLMs ajudam no aprendizado?

Eu gosto de usar em algumas situações:

- Quando não tenho certeza de como começar alguma escrita, normalmente utilizo como uma base geral sem muito conteúdo para então esculpir e adicionar o que eu quero que o texto tenha.
- Correção de erros, principalmente para conteúdos mais específicos. Utilizo quando realmente não faço ideia do porquê algo não funciona.. Claro, sempre anotando e aprendendo o motivo ao invés de só jogar na máquina e esquecer depois de resolvido.

4. Em quais situações elas podem atrapalhar?

Se você deixar ela fazer qualquer trabalho sem nenhuma supervisão ou sequer tentar entender o que ela está fazendo.

5. O que significa utilizar uma LLM como apoio e não como substituta do raciocínio?

Basicamente o que eu disse acima. Deixar a IA fazer o seu trabalho só vai te tornar incapaz de fazer qualquer coisa sem ela.. Não ser dependente, mas utilizar ela como facilitadora para trabalhos mais braçais é onde eu vejo a utilidade desse tipo de tecnologia.

0.7 Reflexão sobre o uso de IA Generativa

1. O que você aprendeu sobre interação com LLMs?

Me foi reforçado sobre como LLMs não devem substituir seu pensamento crítico, mas sim ferramentas para auxílio nos trabalhos que podem sim trazer benefícios se usadas conscientemente.

2. Como a qualidade do prompt influenciou os resultados?

Prompts mais concisos e detalhados trazem menor margem para a IA alucinar e criar respostas sem sentido e não condizentes com o que você queria.

3. Em quais situações a ferramenta foi mais útil?

Dos testes que fiz, a criação de código se saiu melhor do que as explicações.

4. Em quais situações foi necessário verificar ou corrigir respostas?

Quando ela alucinou e tentou responder algo achando que o contexto era outro bem diferente.

5. Você utilizaria essa ferramenta em disciplinas futuras? Por quê?

Sim, com cuidado, revisão e disciplina elas podem sim ajudar a fazer tarefas que seriam mais cansativas, além de auxílio quando não tenho ideia de onde pode ter um erro.